

Verifier-Guided Prompting for Intent-to-Configuration Translation: A Preregistered NetOps Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory trial to test whether constrained prompting plus a deterministic verifier-guided generate→verify→repair loop (one allowed repair) increases deterministic-verifier semantic-correctness when translating operator natural-language intents into low-level device configurations. Design: N=150 synthetic intents sampled from a deterministic generator, shared_initial_candidate generation semantics, model = gpt-5-mini, deterministic validator = deterministic_netops_validator_v5, one initial generation per intent shared across baseline and guarded conditions, guarded pipeline permitted ≤1 repair call. Results (artifact-grounded): baseline = 149/150 (99.333%); guarded = 150/150 (100.0%); paired absolute difference = 0.0066667 (≈0.67 percentage points); exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.00, 0.02] (10,000 resamples, seed = 1729). Provider accounting: completed episodes = 300; model_calls_used = 151; repair-stage invocations = 1; shared_initial_generation = 150. Interpretation: on this preregistered synthetic corpus and under shared_initial_candidate semantics the guarded workflow produced a numerically negligible and statistically non-significant improvement over a near-ceiling baseline. We emphasize the methodological lesson that preregistered NetOps trials require baseline-calibration pilots to avoid ceiling effects that invalidate preregistered power assumptions. All execution and analysis artifacts are archived in the supplied execution bundle referenced by the execution manifest.

I. INTRODUCTION

Natural-language intent interfaces aim to reduce operator burden by letting humans express desired network changes as free text that is automatically translated into executable device configurations. Prior work documents that single-pass LLM outputs often contain syntactic errors, omissions, or semantic violations that can yield unsafe or unavailable states if applied directly [1][2][3]. A commonly proposed defense is a generate→verify→repair architecture in which an LLM produces a candidate, a deterministic verifier checks intent-level invariants or formalized constraints, and an automated repair (or human gate) corrects violations before execution [4][5]. Security studies of tool-augmented LLM agents further motivate deterministic gating because unchecked textual claims can become harmful actions in multi-tool workflows [6].

Research question. After an autonomous literature review and preregistration we posed a confined confirmatory question: does constrained prompting (structured templates with explicit semantic slots) combined with deterministic verifier-guided iterative feedback materially increase verifier-level

semantic correctness of NL→configuration translation versus an unconstrained baseline under a paired design? The trial (study_cand_2026_b2_v1) was preregistered and frozen before any model outputs were observed.

Contributions. (1) A fully preregistered, artifactized paired confirmatory trial measuring deterministic-verifier pass/fail on a programmatically generated synthetic corpus (N=150 paired intents). (2) Artifact-backed outcomes showing that, under shared_initial_candidate semantics and a one-repair budget, the guarded pipeline raised verifier pass rate from 149/150 to 150/150 (+0.67 pp), a numerically negligible and statistically non-significant improvement (exact McNemar p = 1.0; 95% bootstrap CI = [0.00, 0.02]). (3) A methodological recommendation that preregistered NetOps trials include baseline-calibration pilots and explicitly specify generation semantics (shared_initial_candidate vs independent sampling) because semantics materially change the estimand and interpretation.

II. RELATED WORK

Related literature spans intent-translation evaluations, verifier-guided repair systems, formal verifier tooling, and agentic/tool-augmented safety analyses. Multiple recent evaluations of NL→configuration pipelines report that translating underspecified operator language into verifier-sound artifacts is challenging in practice and that prompt design, model competence, and verifier choice strongly influence semantic correctness [1][2][3].

A growing body of work integrates deterministic verification into generation workflows and reports improved correctness for infrastructure-as-code and configuration-repair tasks; reported gains depend on baseline competence, verifier expressiveness, and repair budget (e.g., policy-guided verifier feedback and verifier-in-the-loop fine-tuning) [2][5]. Independent benchmark efforts targeted at configuration repair likewise show generate→verify→repair pipelines can reduce syntactic and semantic errors, but effect sizes vary with sampling semantics (single-shot vs multi-sample generation) and repair allowances [3].

Formal-verification tools (NetKAT/KATch and related symbolic engines) provide concrete oracles for reachability and policy checks and have been proposed as suitability oracles for LLM-generated configurations; these tools are valuable gating layers but vary in protocol coverage and scale, which affects their role as evaluation endpoints [5]. Security analyses

of agentic LLM pipelines demonstrate claim-to-action vulnerabilities and motivate auditable, conservative verifier gating in multi-tool agents [4]. Our study implements a narrowly constrained guarded architecture, preregistration, and paired inference to place experimental bounds on the marginal benefit of a one-repair verifier loop under specific operational semantics.

III. METHODOLOGY

Preregistration and confirmatory design. The study was preregistered as `study_cand_2026_b2_v1` with a frozen analysis plan executed before any model outputs were observed. The primary estimand was the `paired_success_rate_difference_guarded_minus_baseline` evaluated with an exact two-sided McNemar test on deterministic-verifier pass/fail outcomes. The confirmatory sample specified $N=150$ distinct synthetic intents drawn deterministically from a preregistered generator, deterministic inclusion/exclusion rules, a deterministic retry policy for provider timeouts (one retry), and a power target to detect an absolute +15 percentage-point improvement with $\approx 80\%$ power under conservative discordant-pair assumptions. All preregistered elements (estimands, exclusions, multiplicity plan, analysis scripts) were frozen and archived prior to model calls.

Mapping between preregistration labels and executed contrast (required clarification). The preregistration described a conceptual three-arm design: A = baseline free-text prompt, B = constrained-template prompt (constrained-only), and C = constrained prompt + verifier-feedback loop (guarded). The archived execution adapter enforced `shared_initial_candidate` semantics: for each intent exactly one initial sampled candidate was produced and that same candidate was used in both the baseline (A) and guarded (C) paired evaluations. Consequently, the archived confirmatory estimand evaluates the marginal effect of applying at most one repair to the identical initial candidate (C vs A). The constrained-only condition B was not executed as an independent first-pass draw in this run and therefore no confirmatory B-vs-A or C-vs-B contrasts from independent sampling are reported; this mapping is enforced by the adapter and recorded in the execution accounting (`stage_counts = {shared_initial_generation: 150, repair: 1}`). This paragraph clarifies the relation between preregistered labels (A/B/C, H1/H2) and the actually executed paired contrast under `shared_initial_candidate` semantics.

Execution semantics, model, and deterministic validator. Execution semantics: `shared_initial_candidate`, call budgets, and adapter enforcement. The adapter produced exactly one initial model generation per intent (shared across conditions) and allowed at most one additional repair call if the deterministic verifier failed that initial candidate. Provider-accounting recorded `model_calls_used = 151`, `shared_initial_generation = 150`, and `repair = 1`. Declared model: `gpt-5-mini` (provider record: `openai_responses`); `model_configuration.json` records `declared_model_version =`

`"gpt-5-mini"` and `temperature = null (unset)`. No numeric temperature or fixed random-seed was enforced, so independent-session nondeterminism was anticipated in the preregistration and is reported here. Deterministic verifier: `deterministic_netops_validator_v5` (`validator_version = 5.0`) applied the preregistered `full_intent_constraint_validation` rule and returned a boolean pass/fail plus `normalized_configuration` and detailed diagnostics.

Task generator, corpus, and prechecks. Confirmatory tasks were produced deterministically by `deterministic_netops_task_generator_v5` and stratified across four intent families (reachability/isolation, ACL-like changes, VLAN/MTU sequences, uplink switchover). Topologies were limited to verifier-capable small networks (4–32 nodes). Parser transformations and verifier-coverage unit-tests (preregistered) passed before batch execution. Ambiguity-detection and parser-exclusion rules were applied deterministically; no confirmatory exclusions occurred.

Primary analysis and inferential procedures. The primary outcome was deterministic verifier pass (binary). The analysis used an exact two-sided McNemar test on the paired contingency (`n_11`, `n_10`, `n_01`, `n_00`) and a paired bootstrap percentile 95% confidence interval with 10,000 resamples (`bootstrap_seed = 1729`). The preregistered incomplete-pair rule (`complete_pair_primary`) excluded any incomplete pairs from the primary test; no incomplete pairs occurred. Secondary contrasts and multiplicity correction were preregistered but constrained by execution semantics and stage counts; archived analysis artifacts implement the preregistered `paired_binary_analysis_v1` executor and are cited in our artifacts.

Preregistration power invalidation and confirmatory-analysis integrity (required clarification). The preregistered power calculation assumed a substantially lower baseline and targeted detecting +15 pp. The observed `baseline_success_rate = 0.9933` (149/150) produced a ceiling effect that invalidated the preregistered detectable-effect assumption and left very little discordant-pair information for McNemar inference. We confirm explicitly that no post-preregistration changes to the primary confirmatory analysis were performed: the reported analyses follow the archived `paired_binary_analysis_v1` executor and the frozen analysis plan exactly, and the observed power mismatch is reported as a methodological outcome rather than a post hoc redefinition of estimands.

IV. RESULTS

Execution and provider audit. The run completed as planned: `completed_episode_count = 300` and `complete_pair_count = 150`. Provider-call accounting: `model_calls_used = 151`; `completed_hosted_model_call_event_count = 151`; `failed_hosted_model_call_event_count = 0`; `cache_miss_count = 151`. Stage-level counts: `shared_initial_generation = 150` and `repair = 1`. No contamination flags or pre-execution unit-test failures occurred.

Primary numerical outcomes and contingency counts. The paired contingency counts (guarded $\times$ baseline) were: $n_{11} = 149$ (both-pass), $n_{10} = 1$ (guarded-only-pass), $n_{01} = 0$ (baseline-only-pass), $n_{00} = 0$ (both-fail). Per-condition success rates: $\text{baseline_success_count} = 149/150 = 0.9933333333333333$; $\text{guarded_success_count} = 150/150 = 1.0$. Estimated paired absolute difference (guarded - baseline) = 0.00666666666666671 (≈ 0.67 percentage points). Exact McNemar two-sided $p = 1.0$. Paired bootstrap percentile 95% CI = [0.00, 0.02] (10,000 resamples; $\text{bootstrap_seed} = 1729$). These figures are reported verbatim from the archived analysis artifacts (analysis/results.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv).

Repair diagnostics and revision iterations. Only one guarded repair invocation occurred (1/150), consistent with the near-ceiling initial success rate. That single repair converted one initially invalid candidate to a valid configuration under the deterministic verifier. Median revision iterations for guarded episodes = 0 (IQR = 0). Validator traces include `validation_before` and `validation_after` states, `normalized_configuration` strings, `parsed_command_count`, `required_command_count`, `violation_codes`, and a `valid` flag for each episode; representative traces for tasks 000001–000006 are archived and demonstrate varied difficulty patterns while confirming validator behavior.

Representative artifact-grounded example. Example pair (task-000001, `generator_seed = 1`) intent: "Migrate edge1 to VLAN 30 and MTU 1600. For safety, shut edge1 down before changing either VLAN or MTU, then restore it to admin up. Do not modify any other interface." Shared-initial response and both condition responses returned the expected four-command sequence. Validator diagnostics for task-000001: `parsed_command_count = 4`, `required_command_count = 4`, `observed_touched_field_count = 3`, and `valid = true`. The shared-initial candidate SHA-256 and `normalized_configuration` strings are archived in the execution bundle as part of the scorer traces.

Sensitivity and missingness. The preregistered sensitivity analysis treating missing guarded outputs as failures was not invoked because `failed_hosted_model_call_event_count = 0` and `unscorable_episodes = 0`. No exclusions or parse failures occurred and `contamination_summary.csv` is empty.

Interpretation of numeric outcome. On this synthetic corpus and under `shared_initial_candidate` semantics the guarded workflow produced a numerically negligible absolute improvement (+0.67 pp) from an already near-ceiling baseline. The exact McNemar $p = 1.0$ and the bootstrap CI containing zero indicate no statistically supported confirmatory benefit under the preregistered estimand. Importantly, the observed baseline success rate ($\approx 99.33\%$) violated the preregistered power assumption (target detectable effect = 15 percentage points), producing a ceiling effect that invalidated the originally planned sensitivity.

Ceiling effect and implications for preregistration. The preregistered power calculation assumed a substantially lower baseline; observed `baseline_success_rate = 0.9933` left almost no headroom for the guarded repair to show a large absolute improvement. This ceiling effect invalidated the preregistered detectable-effect assumptions (target = 15 percentage points) and reduced the trial’s ability to demonstrate benefit. We therefore recommend that preregistered NetOps trials include a short baseline-calibration pilot (for example, 20–40 tasks sampled from the planned generator and prompt style) to estimate baseline competence and either increase planned N or raise task difficulty before committing to confirmatory sampling. The archived deterministic task generator seeds and representative tasks permit such a pilot: one can sample the same generator with a small pilot, measure `baseline_success_rate`, and then choose confirmatory N or difficulty stratification so that the expected baseline leaves statistical headroom for the target absolute improvement.

Why the synthetic corpus produced a high baseline. The deterministic generator produced tasks with declared difficulty markers (medium, hard, very_hard) and explicit `required_sequence` constraints; inspection of representative traces (tasks 000001–000006) shows many tasks have fully specified required sequences and constrained action spaces (`allowed_fields`, `allowed_touched_fields`) that reduce translation ambiguity. When intents are tightly specified and the verifier coverage matches the required invariants, even a single sampled candidate from a capable LLM often satisfies the verifier. In our run this interaction produced the near-ceiling baseline, illustrating that generator design choices (ambiguity, under-specification, and difficulty-sampling) materially influence the observable marginal benefit of any repair stage.

Semantics, sampling, and estimand trade-offs. The execution semantics (`shared_initial_candidate`) intentionally reduced per-intent sampling variance by using the same initial candidate across conditions; this made the guarded-baseline contrast a pure marginal-repair estimand. Alternative semantics— independent per-condition sampling or ensembles of initial draws—estimate different operational questions. Independent sampling measures expected verifier pass rate when each pipeline draws independently from the model distribution, while multi-draw or multi-repair budgets measure how re-sampling or repeated repairs trade cost for improved correctness. Independent sampling or larger repair budgets typically increase discordant-pair counts and therefore can increase power to detect smaller absolute differences, but at the cost of additional model calls and latency. Practitioners should select sampling semantics that match deployment constraints and incorporate model-call cost and repair-latency budgets into experimental power planning.

Operational affordances beyond pass/fail. Deterministic guarded workflows provide benefits not captured by a single binary endpoint. The validator produces `normalized_configuration` strings, `parsed_command_count`, re-

quired_command_count, and explicit violation_codes that operators can audit; these artifacts accelerate triage and human review even when verifier-pass differences are numerically small. Thus, guarded pipelines can improve operational observability, reduce human triage time, and support auditable change records independent of marginal verifier-pass improvements on constrained corpora.

Threats to validity and future directions. Internal validity: adapter-enforced call budgets and provider audits mitigate cross-condition leakage, and mapping unit-tests reduced parse/translation failures. Construct validity: deterministic validator pass/fail is a proxy for semantic correctness and depends on mapping code and verifier coverage; although mapping unit-tests passed and no parse failures occurred, any uncovered verifier-coverage gap would bias semantic-correctness measurement. External validity: experiments used a single declared model (gpt-5-mini) and synthetic tasks on small topologies (4–32 nodes); results should not be extrapolated to larger, heterogeneous networks or other models. Security and adversarial robustness: this confirmatory trial did not evaluate adversarial prompt-injection; prior work indicates claim-to-action attacks remain an independent concern and defensive evaluation requires separate adversary-driven experiments. Practically, multi-arm experiments that vary repair budgets, sampling semantics, and task ambiguity are necessary to quantify cost–benefit frontiers for operational deployment.

VI. LIMITATIONS

- Single-model constraint: experiments used only gpt-5-mini; results do not generalize across models or sizes.
- Synthetic corpus and scale: tasks were programmatically generated and limited to verifier-capable toy-to-small topologies (4–32 nodes); external validity to production WANs and heterogeneous vendor CLIs is untested.
- Shared_initial_candidate semantics and execution contract limited repair budget to at most one guarded repair call per intent; different generation semantics or multiple-repair budgets may yield different outcomes.
- Ceiling effect and preregistration mismatch: baseline correctness was far higher than the pilot assumptions used for power calculations (planned detectable effect = 15 percentage points), reducing the trial’s ability to demonstrate benefit and illustrating sensitivity to baseline calibration.
- Verifier coverage and specification translation: the study measures deterministic validator pass/fail on the preregistered invariants but does not claim safety guarantees beyond the deterministic validator’s modeled properties.
- Security and adversarial robustness: the experiment did not evaluate adversarial prompt-injection or adversary-driven intents; separate targeted studies are required to assess claim-to-action vulnerabilities in agentic NetOps workflows.

VII. CONCLUSION

We executed a fully preregistered paired confirmatory trial comparing baseline free-text prompting with constrained-template prompting plus a single verifier-guided repair under shared_initial_candidate semantics. On a deterministic synthetic corpus with gpt-5-mini and deterministic_netops_validator_v5, the guarded pipeline increased verifier pass rate from 149/150 to 150/150 (≈ 0.67 pp), a numerically tiny and statistically non-significant difference (exact McNemar $p = 1.0$; 95% bootstrap CI = [0.00, 0.02]). The principal scientific lesson is methodological: preregistered NetOps trials should include baseline calibration to avoid ceiling effects and preserve statistical power. Technically, our artifacts bound the marginal benefit of a one-repair guarded pipeline under shared_initial_candidate semantics; evaluating independent-sampling semantics, larger repair budgets, model diversity, and production-scale topologies remains important future work.

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b39b15ba). Preregistration identifier: study_cand_2026_b2_v1 (the preregistration was frozen prior to observing outcomes and the preregistration record is archived). Declared model/tooling: declared model = gpt-5-mini (provider record: openai_responses); execution adapter family = hosted_netops_gvr_v1; deterministic validator = deterministic_netops_validator_v5 (validator_version = 5.0). Execution semantics mapping (explicit): the preregistration described a conceptual three-arm design A/B/C (A=baseline, B=constrained-only, C=guarded). The archived execution adapter enforced shared_initial_candidate semantics: each intent produced exactly one initial sampled candidate reused across the baseline (A) and guarded (C) paired evaluation; B was not executed as an independent first-pass sampling arm in this run. Provider accounting records this enforcement (stage_counts = {shared_initial_generation: 150, repair: 1}; model_calls_used = 151). Runtime sampling parameters: model_configuration.json records temperature = null (unset); no numeric temperature or fixed random-seed was enforced, so independent-session nondeterminism was anticipated. Autonomy and human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the autonomous system performed literature review, study selection, preregistration, code generation, experiment execution, analysis, manuscript drafting, autonomous peer review, and revision. No human manually edited technical content, analyses, figures, captions, or references after the autonomy lock. Confirmatory-analysis integrity: the preregistered analysis plan targeted detecting +15 percentage points but the observed baseline_success_rate = 149/150 ($\approx 99.33\%$) produced a ceiling effect that

invalidated that power assumption; we confirm that no post-preregistration changes to the primary confirmatory analysis were made and that the reported analyses follow the archived `paired_binary_analysis_v1` executor. Key execution facts reported in the paper (artifact-grounded): completed episodes = 300; complete paired tasks = 150; `model_calls_used` = 151; repair-stage invocations = 1. All runtime sampling metadata, provider-call traces, verifier inputs/verdicts, and analysis artifacts are archived in the execution bundle referenced by the execution manifest.

REFERENCES

- [1] <https://doi.org/10.1002/nem.2313>
- [2] <https://doi.org/10.1145/3786583.3786898>
- [3] <https://arxiv.org/abs/2604.22513>
- [4] <https://doi.org/10.1145/3605764.3623985>
- [5] <https://doi.org/10.1145/3656454>
- [6] <https://doi.org/10.1002/widm.70111>